Inlämningsuppgift 1 – Integrera shoppen med omvärlden

Du kopplar din egen webbshop mot externa tjänster: sök, produktimport, betalning, molnlagring och frakt. Uppgiften är individuell, bedöms med IG/G och lämnas in senast söndag 1 november 2026 kl. 23:59.

Översikt

Uppgiften täcker Block A (lektion 1–7, vecka 41–44). Du bygger vidare på samma kodbas som i Uppgift 2, så det du gör nu följer med resten av kursen.

Läranderesultat	1, 3 och 5 (se bedömningsmatrisen)
Betygsskala	IG/G
Arbetsform	Individuell. Du får diskutera med andra, men koden och texten ska vara din egen.
Presenteras	L3, måndag 12/10
Inlämning	Söndag 1/11 kl. 23:59
Kodbas	Din egen webbshop från backendkursen, eller startrepot om din egen inte fungerar
Deploy	Shoppen ska vara deployad, t.ex. på Vercel eller Render. Bedömningen görs på den deployade shoppen.
Tjänster	Bara testlägen och gratis sandlådor. Du skapar egna testkonton, och inga riktiga pengar eller avtal behövs.
Bedömning	Rättning sker alltid inom 2 veckor efter deadline
Redovisning	Hela projektet (Uppgift 1 och 2) redovisas fredag 20/11 kl. 13–17

Läranderesultat (kursplanen):

- **LR 1:** Förklara principer för integration av tredjepartssystem och deras användningsområden i webbapplikationer.
- **LR 3:** Integrera tredjepartssystem och REST-baserade API:er, inklusive betalning, lagring och logistik, i webbapplikationer.
- **LR 5:** Implementera och anpassa sök-API:er samt hantera och tolka standardiserade dataformat (JSON, XML, CSV) för att skapa funktionella digitala tjänster.

Gemensamma krav för alla integrationer

Kraven gäller varje extern tjänst i uppgiften: leverantörens filer, betalning, molnlagring och frakt.

1. **Anrop via backend.** React anropar bara din egen Express-server, aldrig en extern tjänst direkt. Hemliga nycklar finns bara på servern.
2. **En adapter per tjänst.** All kod som vet hur en tjänst fungerar (URL, nycklar, format, felkoder) ligger i en egen fil, t.ex. `integrations/paymentAdapter.ts` (eller .js om din shop är skriven i JavaScript). Adaptern returnerar shoppens format, inte leverantörens rådata.
3. **Nycklar i `.env`.** `.env` finns i `.gitignore` och har aldrig committats. `.env.example` visar alla variabler som behövs, utan riktiga värden. I den deployade shoppen ligger nycklarna i plattformens miljövariabler.
4. **Timeout och felhantering.** Varje anrop har en timeout. Om tjänsten är nere eller svarar med fel får användaren ett begripligt meddelande, och shoppen kraschar inte.
5. **Validera extern data.** Data från andra system (JSON, CSV, XML) kontrolleras innan den används eller sparas, t.ex. med Zod. Trasiga poster hoppas över eller ger ett tydligt fel.

Labb 1–5: integrationerna

Labbarna kommer i samma ordning som lektionerna. Varje labb avslutas med ett test: **Klart när**. Alla fem labbarna krävs för G. Kravspecen för varje labb ges ut på lektionen där konceptet gås igenom.

Labb 1 – Import av produktdata (L3, LR 5)

Kravspecen finns i Labb 1, som ges ut på L3.

Labb 2 – Sök-API (L4, LR 5)

Kravspecen ges ut som Labb 2 på L4.

Labb 3 – Betalning (L5, LR 3)

Kravspecen ges ut som Labb 3 på L5.

Labb 4 – Produktbilder i molnlagring (L6, LR 3)

Kravspecen ges ut som Labb 4 på L6.

Labb 5 – Frakt (L7, LR 3)

Kravspecen ges ut som Labb 5 på L7.

README och reflektion (LR 1)

README i repots rot visar att du förstår *varför* shoppen är byggd som den är, inte bara att den fungerar. Den innehåller två delar.

1. Kom igång

- Länk till den deployade shoppen.
- Steg för att starta klient och server lokalt.
- Vilka testkonton som behövs och var nycklarna hamnar (hänvisa till `.env.example`).
- Eventuella extra steg för att testa integrationerna lokalt.

2. Reflektion (ca 400–800 ord)

Svara med exempel från din egen shop:

1. Vilken integrationsmodell använder varje tjänst (REST, webhook, redirect, fil eller SDK), och varför passar just den?
2. Varför går anropen till externa tjänster via Express och inte direkt från React?
3. Förklara kort för varje tjänst du har integrerat varför det är bättre att integrera den än att bygga samma sak själv. Vad skulle det kräva att bygga den själv?
4. Vad händer i shoppen om var och en av tjänsterna är nere, och hur har du hanterat det?
5. Vilka risker för beroendet med sig för minst en tjänst, t.ex. kostnad, ändrade villkor eller persondata (GDPR)?

Bedömningsmatris

För G ska alla tre läranderesultaten vara uppnådda. Saknas något av dem blir betyget IG.

Läranderesultat	För G krävs	Bedöms i
LR 1 – Förklara principer för integration av tredjepartssystem	Reflektionen besvarar alla fem frågorna med exempel från den egna shoppen. Integrationsmodellerna är rätt identifierade och valen är motiverade.	README och reflektion
LR 3 – Integrera tredjepartssystem och REST-API:er för betalning, lagring och logistik	Betalning, molnlagring och frakt uppfyller sina *Klart när*. Webhooken är signaturverifierad, och pris och frakt räknas på servern. De gemensamma kraven gäller för alla tre.	Labb 3–5, gemensamma krav
LR 5 – Sök-API:er och dataformaten JSON, XML och CSV	Importen tolkar både CSV och XML rätt, validerar och gör upsert. Sök-API:et har fritext, filter, sortering och paginering och validerar parametrarna. Extern data valideras innan den används.	Labb 1–2, gemensamma krav

Du får använda AI-verktyg, men du ska kunna förklara all kod i ditt projekt vid redovisningen.

Komplettering: Vid IG får du besked om vad som saknas. Enligt kursplanen har du rätt till komplettering inom två månader efter det ordinarie tillfället.

Uppgiften presenteras på L3 och varje lektion därefter ger underlag till en labb. Skriv README löpande. Vecka 44 är det handledning hela onsdagen och torsdag 13–17 i stället för fredag.

Inlämning

Lämna in två länkar på lärplattformen senast söndag 1/11 kl. 23:59: ditt GitHub-repo och din deployade shop. Är repot privat bjuder du in läraren som medarbetare.

- Bedömningen görs genom att testa den deployade shoppen och granska koden i den senaste commiten före deadline. Senare commits räknas bara vid komplettering.
- Allt i *Klart när* – testet i slutet av varje labb – ska gå att göra i den deployade shoppen. Det som inte fungerar där räknas som inte uppfyllt.
- Länken till shoppen ska visa versionen från deadline tills rättningen är klar. Fortsätter du med Uppgift 2 kan du t.ex. länka till Vercels fasta URL för just den deployen, eller arbeta på en annan gren än den som deployas.
- Har shoppen admininloggning skriver du ett testkonto för läraren i kommentaren vid inlämningen, aldrig i repot.
- Håll shoppen och databasen igång tills rättningen är klar. Gratisprojekt i t.ex. Supabase pausas efter en vecka utan aktivitet.
- Du fortsätter i samma repo med Uppgift 2.

Checklista före inlämning

- [ ] Import av både CSV och XML uppfyller *Klart när* i Labb 1
- [ ] Sök-API:et och sökningen i React uppfyller *Klart när* i Labb 2tele
- [ ] Ett testköp gör ordern betald via webhooken (Labb 3)
- [ ] Produktbilder laddas upp och visas från molnlagringen (Labb 4)
- [ ] Fraktpriset räknas på servern och ingår i betalningen. Kunden kan välja ombud och ser beräknad leveranstid (Labb 5)
- [ ] Varje extern tjänst har en egen adapter, timeout och felhantering
- [ ] `.env` finns inte i repot eller dess historik, och `.env.example` är komplett
- [ ] README har *Kom igång* och reflektionen
- [ ] Shoppen är deployad, och du har själv gjort testet i slutet av varje labb (Labb 1–5) i den deployade shoppen